

Track C: Calibration, Scale & Backend API

Team: 1 human + 1 AI agent **Owns:** `/track_c_api` **Starts Day 1** against Track B's mock `mesh.obj` in `/shared/sample_jobs/job_0001` — do not wait for Track B's real pipeline.

1. What this track delivers

- A closed-form scale factor calculation: given one known real-world reference length in the scene (e.g. a ruler or an object of known size the human photographed), compute `scale.json` and apply it to the mesh so its units become meters.
- A FastAPI backend exposing:
 - `POST /jobs` — accepts a photo, kicks off the pipeline, returns a job id
 - `GET /jobs/{id}` — returns status per the contract's job-status schema
 - `GET /jobs/{id}/result` — returns the final scaled mesh
- MVP scope: **synchronous, single job at a time**. Background jobs and multi-user concurrency are explicitly deferred (see §4).

2. Who does what, and why

What the agent does: - Writes the closed-form scale calculation (ratio of a known real length to its measured length in the unscaled mesh). - Writes the FastAPI endpoints, request/response schemas, and error handling. - Explains failure modes before they happen — e.g. what breaks if the reference object isn't visible in the mesh, or if two tracks' file formats drift apart — so the human knows what to test for. - Reviews the human's test requests/curl commands and interprets error tracebacks.

What the human does: - Physically measures and photographs the real-world reference object used for calibration (a tape measure against a real object — this cannot be done by the agent). Reports the true length back precisely. - Runs the API locally, fires real requests at it (via curl, Postman, or a simple script), and reports back real status codes and response bodies — the agent can write a client but only the human's environment tells you if the server actually starts and responds. - Makes the call on which single reference-object measurement method to standardize on across all hero images (consistency matters more than precision for MVP), and documents it for Track D's accuracy report. - Decides, with the agent's input, whether a given failure is worth fixing this week or belongs on the post-MVP backlog — a scope call, not a technical one.

Why this split: ground-truth measurement is physical and can only come from a human with a tape measure and a camera; running a live server and confirming it actually serves real requests requires a human's machine and judgment about what "working" means in practice. The agent's contribution is writing correct, fast code and reasoning about edge cases before the human hits them.

3. Day-by-day

Day	Task
1	Agent scaffolds FastAPI skeleton + scale-factor math against the mock mesh. Human confirms the server starts and <code>/jobs</code> returns a job id.
2	Human measures and photographs 2-3 real reference objects; agent wires the scale calculation to real numbers; human verifies the scaled mesh

	dimensions look plausible (e.g. “that object really is about 30cm”).
3	Wire the endpoints to call Track A → Track B in sequence (still synchronous); human runs an end-to-end local test on the mock chain, then swaps in real Track A/B output as it becomes available.
4	Run all hero images through the full synchronous pipeline via the real API; human documents actual response times; write “what we built vs. contract” note.
5	Integration day — this track is the backbone everyone else plugs into; human+agent pair should expect to be the most active pair in the room.

4. Explicitly out of scope for this week

Background job processing and multi-user job isolation are real robustness improvements — they’re the *last* item on the post-MVP upgrade backlog on purpose (accuracy matters more than infra polish for a first demo). Keep this week’s API strictly synchronous and single-job; don’t let it grow scope mid-week.

5. Definition of done

- `POST /jobs` → `GET /jobs/{id}` → `GET /jobs/{id}/result` works end-to-end on at least 3 real hero images, through the real Track A/B pipeline (not mocks).
- `scale.json` produced correctly for each, using a documented, consistent reference-measurement method.
- Response times from a real local run recorded and reported.